Grokking in LLM Pretraining? Monitor Memorization-to-Generalization without Test

Ziyue Li[1], **Chenrui Fan**[1], **Tianyi Zhou**
[1]Department of Computer Science, University of Maryland, College Park
litzy619@umd.edu, cfan42@umd.edu, tianyi.david.zhou@gmail.com

Abstract

This paper presents *the first study of grokking in practical LLM pretraining*. Specifically, we investigate when an LLM memorizes the training data, when its generalization on downstream tasks starts to improve, and what happens if there is a lag between the two. Unlike existing works studying when a small model generalizes to limited and specified tasks during thousands epochs' training on algorithmic data, we focus on a practical setting for LLMs, i.e., near single-pass pretraining of next-token prediction on a cross-domain, large-scale corpus, and generalization on diverse benchmark tasks covering math/commonsense reasoning, code generation, and domain-specific retrieval. Our study, *for the first time, verifies that grokking still emerges in pretraining mixture-of-experts (MoE) LLMs*, though different local data groups may enter their grokking stages asynchronously due to the heterogeneity of their distributions and attributions to others. To find a mechanistic interpretation of this local grokking, we investigate the dynamics of training data's pathways (i.e., expert choices across layers in MoE). Our primary discovery is that *the pathways evolve from random, non-smooth across layers, instance-specific to more structured and transferable across samples*, despite the converged pretraining loss. This depicts a transition from memorization to generalization. Two novel metrics are developed to quantify these patterns: one computes the pathway similarity between samples, while the other measures the consistency of aggregated experts between subsequent layers for each sample. These training data based metrics induce near zero cost but can faithfully track and monitor the generalization of LLMs on downstream tasks, reducing reliance on costly instruction tuning and benchmark evaluations. We also ground our findings in a theoretical analysis of one-layer MoE, showing that more structured pathways improve the generalization bound.

1 Introduction

Large language models (LLMs) with Transformer [30] architectures have demonstrated that pretraining for a general-purpose task such as autoregressive prediction on a large corpus can gain powerful generalization across diverse downstream tasks and domains [3, 29, 2, 36]. Capabilities such as prompting [32], reasoning [10, 1], and in-context learning also emerge during training. However, a counter-intuitive "grokking" [24, 22, 28, 18] has been widely observed in training Transformer models: the generalization starts to improve or even sharply rises long after training loss converged or plateaued—an indicator of overfitting in conventional machine learning. While grokking indicates a transition from memorization to generalization [11], it challenges our fundamental understanding of training dynamics and the underlying mechanism of generalization.

Despite recent attempts to explain the generalization induced by grokking, they have been largely confined to small models trained for hundreds to thousands of epochs on algorithm-synthesized small-scale data with generalization to limited and highly specified tasks [24, 15, 22, 19, 12, 5, 25]. Instead, LLMs' pre-training often takes only one epoch on a web-scale corpus of heterogeneous,

Preprint. Under review.

cross-domain data that usually contains noise, imbalance, and redundancy [3, 36]. Hence, LLMs' learning dynamics can vary drastically across different data, leaving whether grokking still emerges in LLM pretraining an open problem. It is also unclear whether and when different data are memorized with converged loss, and, whether they are memorized at the same time. Moreover, without the repeated replay of the same data and a decrease in loss, the conversion from memorizing training tokens to generalizing on downstream tasks remains mysterious: When does the model start to generalize to different tasks? Is memorization a prerequisite? How does generalization performance change after memorization?

Unfortunately, these questions cannot be answered by the dynamics of pretraining loss since the generalization performance can still vary after the loss plateaued, as observed in previous studies of grokking [24, 22]. For LLMs, it is also practically expensive to monitor its generalization performance during pretraining, since the model has to be finetuned to gain instruction-following capability before being evaluated on downstream tasks. Hence, developing an efficient-to-compute metric to accurately track the generalization can offer critical insights on better understanding and improving the pretraining of LLMs.

Main Contributions In this paper, we investigate the training dynamics and grokking in LLM pretraining for a mechanistic interpretation of its emergent generalization on downstream tasks. Our study is conducted on the open-sourced pretraining checkpoints of a 7B mixture-of-experts (MoE) LLM, OLMoE [21]. While our study mainly focuses on OLMoE, its pretraining and evaluation setups are common among other LLMs. For the first time, we verify the existence of local grokking during LLM pretraining. However, unlike the previously observed global and synchronous grokking for most data [24, 19, 22], *LLM memorizes domains/groups of training data at different training steps and takes varying amounts of steps to generalize to related benchmark tasks.* The starting time and lasting steps of such local grokking vary across data groups. As a result, generalization performance is usually unstable during the earlier pretraining stages but starts to steadily improve once sufficient data have been memorized. This local difference also reflects data difficulty: the later memorized data group often takes longer to generalize.

Another primary challenge is to explain how the continual pretraining after memorization completed leads to the delayed sharp rise of the generalization performance. To investigate the mechanism of memorization-to-generalization transition, we explore internal states of LLMs track their major changes after loss converges. Specifically, we study how the pathways (the expert choices across layers for training samples) in OLMoE change. We introduce two metrics to measure their complexity how it changes over time: (1) the edit distance between different samples' pathways; and (2) the consistency of selected experts' embedding between consecutive layers for each sample.

Our empirical analysis on four domains reveals a prominent trend of the pathway complexity during grokking: the edit distance declines and the consistency increases, despite the converged training loss and more data being learned. This change in pathway complexity reflects a transition to smarter memorization: it keeps discovering more transferable knowledge across samples to encode data more efficiently. *It explains why generalization improves with plateaued training loss: the model keeps finding more generalizable and shareable structures to memorize training data.* Additionally, we provide a theoretical explanation relating the pathway complexity to a generalization bound of a one-layer MoE. Practically, the two metrics show strong correlations with the evaluation results on standard benchmarks after instruction finetuning. Therefore, they provide a zero-cost, finetuning and benchmark evaluation-free tool to monitor generalization during pretraining, which is critical to LLM developers and practitioners. These discoveries take the first step toward bridging the gaps of understanding grokking and memorization-to-generalization transition in LLM pretraining. Our key findings and contributions can be summarized as:

- Grokking still occurs during the near single-pass pretraining of practical-scale LLMs, but it is local and asynchronous for different data groups, unlike global grokking for all data in previous works.
- Grokking's memorization-to-generalization transition can be explained by the dynamics of pathways in MoE: pathway similarity between samples and consistency across layers increase. A theoretical connection is also built between the pathway complexity and a generalization bound.
- The two metrics we developed to measure pathway complexity provide a finetuning and evaluation-free tool with almost zero cost to monitor the generalization during LLM pretraining.

2 Related Work

Previous studies on grokking have primarily focused on relatively small models and simplistic tasks. Grokking was first identified by Power et al. [24], who observed the phenomenon using a two-layer transformer trained on a small-scale algorithmic dataset. Subsequent investigations have extended these findings to shallow transformers and densely connected networks [22, 23, 19, 16, 6], and CNNs [12]. Notably, all aforementioned studies employ multi-epoch training paradigms, which differ substantially from the one-pass training approach typical in LLM pretraining. Although Lv et al. [18] examined grokking behavior using a 162M parameter transformer pretrained on 40 billion tokens for a copying task, the scale of their model, dataset, and the simplicity of the task remain distant from practical, real-world scenarios. In contrast, our study is the first to investigate grokking in a 7-billion parameter LLM across diverse and practical tasks, including mathematical reasoning, code generation, commonsense understanding, and domain-specific knowledge retrieval.

Several works have focused on revealing the mechanisms behind grokking behavior. Liu et al. [15] demonstrated in a simplified setting that grokking results from structured representations emerging during training. Nanda et al. [22] further revealed that grokking can be attributed to the gradual amplification of structured mechanisms encoded within model weights. Wang et al. [31] showed that even small Transformers trained from scratch on synthetic reasoning tasks exhibit a grokking-driven emergence of implicit reasoning, where generalization arises only after extended overfitting and internal structure formation. Using a single-layer ReLU network, Merrill et al. [19] associated grokking with subnetwork sparsity. Meanwhile, Humayun et al. [12] observed that decision boundaries become smoother during the grokking phase in CNN classification tasks. Beyond interpretability-focused research, Notsawo Jr et al. [23] identified oscillatory patterns in early training loss as predictors of subsequent grokking phenomena. Distinct from these works, our research provides an explanation of grokking within significantly larger LLMs through an interpretative analysis of MoE architectures. Additionally, we introduce practical indicators for monitoring generative behavior throughout LLM pretraining.

3 Grokking (Delayed Generalization) in LLM Pretraining

While most existing works study grokking of small models on specified algorithmic tasks, investigating similar phenomena in LLM pretraining poses several new unique challenges, which inherently motivate several designs in our experimental settings.

- **Training-test evaluation gap**: most LLMs are trained for the next token(s) prediction while evaluated on diverse benchmark tasks, making the conventional comparison between training and test loss/accuracy invalid. Hence, we need to adapt the definition of grokking to the LLM setting.
- **Heterogeneous, web-sourced, cross-domain training data**: While previous grokking studies focus on clean training data with the same distribution/format, most LLMs are pretrained on heterogeneous, cross-domain, web-sourced data that contain redundant, contaminated, or imbalanced samples. Hence, their loss may converge at different rates and pose distinct effects on downstream tasks. To address these challenges, we carefully filter and group the data in our analysis.
- **Generalization to diverse downstream tasks**: Previous studies of grokking focus on generalization to highly-specified and limited tasks, while practical LLMs are expected to generalize to arbitrary tasks or domains via prompting. Hence, LLMs' generalization behaviors may vary drastically across different downstream tasks/benchmarks and difficulty levels during pretraining.
- **Near single-pass pretraining**: Most LLMs' pretraining only takes approximately one epoch, so the model has been trained only once on each sample, and its training loss may even converge before visiting later data. However, such "memorization" of unseen data is not caused by overfitting or repeated replay. Instead, it reflects the interpolation or composition of learned data, which might be counted as "generalization" in previous works but differs from the emergent generalization of LLMs on new tasks.

3.1 Study Training Dynamics of LLM Pretraining

Memorization is measured by the pretraining objective. Next token prediction (NTP) is the most widely adopted pretraining objective for LLMs. Let the pretraining corpus be D_{train}, it aims to

minimize the negative log-likelihood (NLL) of each token $x_{i,j}$ given previous tokens $x_{i,<j}$ in a text sequence $x_i = (x_{i,1}, \ldots, x_{i,|j|})$ drawn from D_{train}:

$$\ell_i(\theta) = -\frac{1}{|x_i|} \sum_{j=1}^{|x_i|} \log p_\theta(x_{i,j} \mid x_{i,<j}) \,. \tag{1}$$

Since $\ell_i(\theta)$ directly measures how the model memorizes each token in x_i, we will track it in the pretraining process and identify the sample with consistently small and converged loss across checkpoints as memorized. Specifically, for a sample x_i with loss $\ell_i(\theta_t)$ at pretraining step t, we identify it as *memorized* since step t_i^* if $\ell_i(\theta_t) \leq \varepsilon$ and $|\ell_i(\theta_t) - \ell_i(\theta_T)| \leq \delta$ for all $t \geq t_i^*$, where ε is a small threshold, δ enforces stability over time, and T indexes the final pretraining checkpoint. In Appendix E.3, We verify that varying ε and δ does not change the post-memorization behavior analyzed in Section 4.

Generalization is measured on standard benchmarks after instruction tuning. As foundation models with various emergent capabilities, LLMs are expected to generalize to an open set of unseen tasks through simple prompting/instructions. Hence, their generalization performance is usually measured across standard benchmarks defined on diverse downstream tasks. To equip a pretrained LLM θ with the instruction-following capability, an instruction tuning algorithm $\mathcal{A}_{\text{IT}}$ further finetunes θ on a dataset D_{IT}. The generalization performance on benchmark-i is then measured by

$$\mathcal{G}_i = \text{Eval}_i(\theta_{IT}, D_i), \theta_{IT} = \mathcal{A}_{\text{IT}}(\theta, D_{\text{IT}}), \tag{2}$$

where D_i is the dataset in benchmark-i and Eval_i defines the corresponding evaluation metric, which varies across benchmarks for different types of tasks. For example, accuracy is usually used in multi-choice QA tasks while BLEU/ROUGE scores are used in summarization and generation tasks. For code generation tasks, the generalization performance is usually measured by unit tests, where a test case is considered correct if the generated code can pass the unit tests.

Grokking refers to delayed generalization improvement after memorization is completed. Following the same spirit of previous works on grokking, we mainly focus on the phenomenon that $\mathcal{G}_i$ starts to surge long after $\ell_i(\theta)$ has converged/plateaued. We also use the training loss to track memorization as previous works. However, the test loss adopted by previous grokking analysis is no longer an ideal metric for LLM generalization, since it focuses on token-level matching to ground truths while downstream tasks focus on semantic-level matching, and some of them do not provide ground truths (e.g., code generation).

Experimental Setup We equidistantly sample 10 checkpoints from the OLMoE pretraining trajectory. To evaluate memorization, we probe OLMoE on samples drawn directly from its pretraining corpus D_{train}, ensuring fidelity to the model's original distribution. We evaluate generalization on widely used benchmarks spanning four domains: *math*, *code*, *commonsense QA*, and *domain-specific QA* (10k pretraining and 10k test samples per domain; see Appendix A). To exclude contaminated benchmark samples from the generalization analysis, we apply the Min-K%++ membership inference method [35] to identify them. Moreover, we mainly focus on the samples whose model predictions become consistently correct before the pretraining ends, and analyze when this generalization emerges and how long it persists. For downstream tasks, we apply lightweight LoRA-based instruction tuning $\mathcal{A}_{\text{IT}}$ on D_{IT} to equip checkpoints with basic instruction-following capability while preserving pretrained capabilities (Appendix B).

3.2 Asynchronous Memorization, Delayed Generalization, & Local Grokking

To understand when memorization unfolds during pretraining and how it affects generalization, Figure 1 contrasts the number of memorized training samples (using the criterion defined above) at each pretraining checkpoint with benchmark performance for each domain. It shows that (1) training samples are memorized at different pretraining steps; (2) generalization typically follows memorization with a lag: benchmark performance starts to achieve stable gains after sufficient data has been memorized. An interesting discovery is that the lag length varies across domains: generalization leap on math and coding tasks requires memorizing many more samples than commonsense and domain-specific QA tasks.

These observations reveal the complex training dynamics of LLMs caused by training data heterogeneity and their uneven attributions to different benchmark tasks. It implies that the widely studied

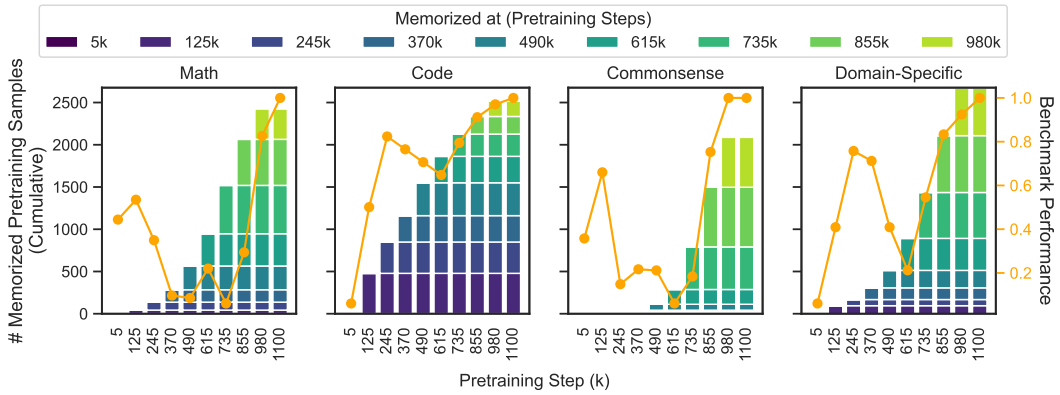

Figure 1: **Memorized training samples at different pretraining steps and the generalization performance** on standard benchmarks for each domain. It shows asynchronous memorization of different data in each domain and a delayed generalization leap after memorizing a certain amount of data.

global grokking—a synchronous memorization of most training data and a delayed contemporary generalization leap on most test samples—cannot be found in LLM pretraining. That being said, the memorization of certain data is plausible to impose a long-term, delayed effect on the generalization to specific downstream tasks. To investigate such local dynamic patterns, we group data in each domain by their memorization steps t_i^* and particularly focus on the early-memorized groups, as they leave more checkpoints after t_i^* to study the subsequent dynamics of generalization. Accordingly, we group samples in benchmarks by the training steps since when their predictions become consistently correct. To enable an analysis of how memorization drives downstream generalization performance, we pair each training data group with its most relevant benchmark samples via bipartite graph matching (by Hungarian algorithm [13]), with the training-test data similarity defined in the embedding space of SentenceTransformer [27]. Additional implementation details of the pairing procedure are provided in Appendix B.1.

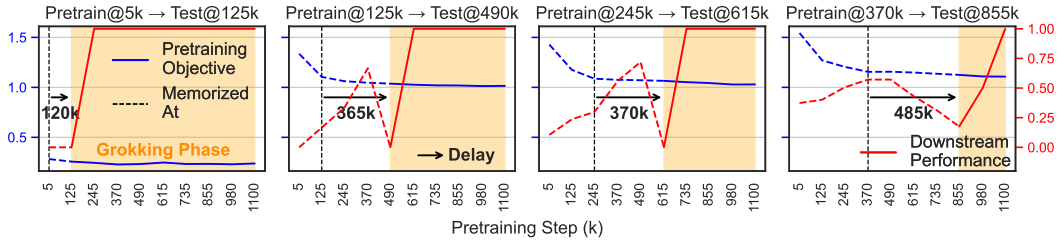

Figure 2: **Pretraining objective (blue) and benchmark performance (red) across four paired training-test data groups from the math domain.** Pretrain@p → Test@q denotes a paired training–test group, where Pretrain@p is the training subset whose samples reach stable memorization at step p, and Test@q is the benchmark subset whose accuracy stabilizes at step q. These pairs are formed using embedding-based Hungarian matching, as detailed in Appendix B.1. We observe local grokking on each pair: a delayed generalization leap on benchmark tasks after the pretraining objective plateaued. From left to right, more difficult data memorized (converged) later associate with a longer delay towards grokking.

Figure 2 visualizes the memorization and generalization on four groups: all training-test pairs exhibit a delayed generalization leap on benchmark tasks after the pretraining objective stably converged on the training data. Hence, the training objective cannot monitor or predict the generalization performance. Moreover, earlier-converging groups generalize sooner after loss converged, while later-converged groups associate with substantially longer delays. Notably, this delay scales with data difficulty, which affects not only the number of steps taken to memorize the data but also the duration of memorization to generalization transition.

Remaining Question This local grokking indicates that the model's internal states, except its pretraining loss, might track generalization or reflect the transition better. This raises a fundamental question for understanding the emergence of capability in LLM: *What internal state changes within an LLM signify the emergence of generalization?* Investigating this problem can provide critical insights to uncover the mechanisms of memorization-to-generalization transition and monitor the pretraining progress in practice. To this end, we probe LLMs' internal state dynamics to address the challenge.

4 Monitor Memorization-to-Generalization Transition by Training Dynamics of Routing Pathways

As shown in Section 3, generalization in LLMs emerges well after memorization, suggesting that internal reorganization underlies this transition. Such dynamics are difficult to track in dense architectures, where all neurons are simultaneously involved. MoEs simplify this process: by organizing computation into experts, each input activates only a subset of experts, forming a discrete *routing pathway* across layers (Figure 3(a)). These pathways reveal how computation is allocated and reorganized during pretraining, providing a mechanistic view of the memorization-to-generalization transition.

In this section, we analyze the evolution of *routing pathway* during pretraining and introduce two metrics to quantify these changes (Figure 3): (i) the similarity of pathways across different inputs (Section 4.1), and (ii) the consistency of a single input's pathway (Section 4.2). Importantly, all metrics are computed on pretraining samples that have already been memorized, i.e., those whose training objective has stabilized at low values beyond specific checkpoints, and trace how their internal routing continue to evolve from memorization to generalization. We then assess these metrics as robust generalization monitors (Section 4.3) and conclude by establishing a spectral framework connecting routing geometry to generalization bounds (Section 4.4).

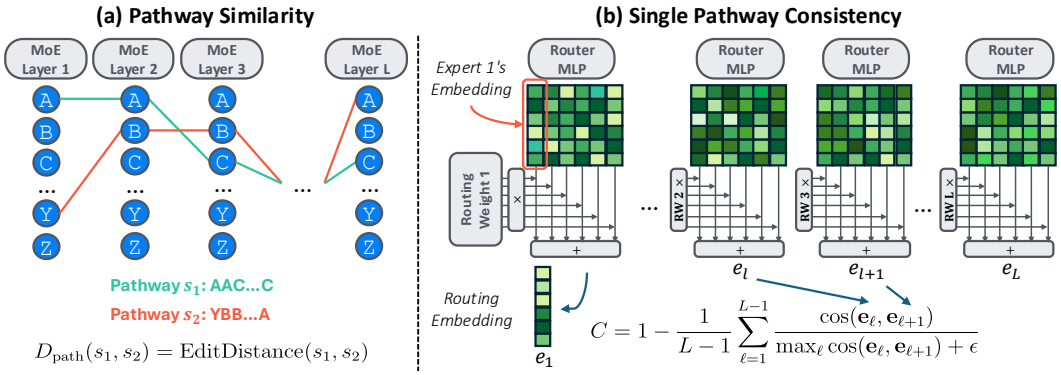

Figure 3: **Pathway Complexity Metrics to Monitor Grokking.** (a) Pathway similarity between samples is measured by edit distance on their sequences of expert choices across layers. (b) Pathway consistency quantifies the smoothness of expert transitions between subsequent layers for the same sample by cosine similarity of their weighted expert embeddings.

4.1 Pathway Distance and Knowledge Sharing between Training Samples

Building on prior findings that grokking coincides with the emergence of *structured internal mechanisms* [15, 22, 31], we hypothesize that as the model begins to generalize, it transitions from highly individualized, input-specific computation toward more *shared, structured computation across related inputs*. This shift should manifest as increasing **pathway similarity**—the convergence of routing patterns across similar samples—indicating that the model is developing common mechanisms for solving related tasks.

We test this by examining how the similarity of expert pathway evolves during pretraining. Each input's pathway is the ordered sequence of selected experts through the L MoE layers. Specifically, for input x_i, we define its pathway s_i as: $s_i = \text{concat}(e_1^{(i)}, e_2^{(i)}, \ldots, e_L^{(i)})$, where $e_\ell^{(i)}$ denotes the ordered list of expert indices at layer ℓ, obtained by ranking experts according to the mean routing

weights across all tokens for input x_i. To determine which experts are included, we first rank experts based on their routing weight and then select top-ranked experts until their cumulative routing weight exceeds a predefined threshold. The weight threshold selects the experts that actually drive each input's computation, rather than picking a fixed number of experts. In our main analysis, we use a cumulative routing-weight threshold of 0.7, and Appendix E.1 shows that the observed pathway trends remain consistent when varying this threshold. The selected experts are then concatenated as a comma-separated string (e.g., `'3,1,5'`) which are later joined across layers with hyphens (e.g., `'3,1,5-...-9,1'`).

We use the Levenshtein edit distance $D_{\text{path}}(s_i, s_j) = \text{EditDistance}\big(s_i, s_j\big)$, which counts the minimum insertions, deletions, or substitutions required to align two pathways, to measure pathway similarity. This metric captures both local deviations in expert choice and global shifts in pathway structure, reflecting differences in selection, length, and ordering. We monitor the average pairwise distance across samples throughout pretraining to track the emergence of shared routing.

Overall pattern. Our analysis, shown in Figure 4, reveals a clear trajectory in pathway dynamics. Early in pretraining, most inputs share nearly identical routes, producing low edit distance D_{path}. As the model memorizes individual inputs, pathways diverge and D_{path} rises. Crucially, D_{path} decreases after memorization, which indicates that semantically related inputs then converge into similar routing sequences, signaling the **emergence of shared pathways** that support generalization.

Layerwise pattern. While prior work [17] reports static depth-dependent specialization in trained MoEs, our results show that the memorization-to-generalization transition is also dynamically depth-dependent during pretraining (Figure 5). Shallow layers show the steepest decline in D_{path} after memorization, indicating rapid convergence to shared pathways. Deeper layers exhibit smaller reductions, and the final layer even shows an increase in D_{path} post-memorization. These patterns suggest a depth-specific reorganization: early layers consolidate universal representations, while later layers retain or enhance routing flexibility, balancing shared computation with task-specific specialization.

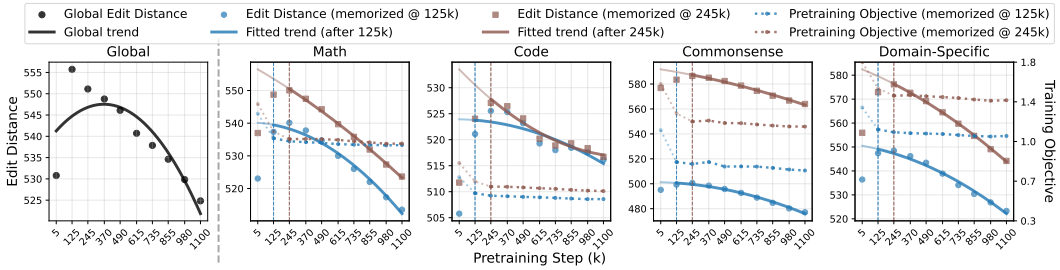

Figure 4: **Pathway edit distance and pretraining objective of two data groups** (memorized at $125k$ and $245k$ steps) during pretraining across four domains. The global panel (leftmost) fits the overall quadratic trend over all domains, while domain panels fit separate trends after the corresponding convergence step. Despite early training objective plateauing, pathway edit distance continues to decline, indicating a declining complexity of internal memorization.

4.2 Pathway Consistency and Complexity for Single Samples

We next analyze how the routing complexity of individual inputs evolves during pretraining. Our hypothesis is that increasingly streamlined and consistent expert selection across layers reflects the reuse of shared pathways, signaling the emergence of generalizable processing strategies. To capture this, we define **single-sample pathway consistency**, which measures the smoothness of expert transitions across consecutive layers.

In a typical MoE, the router at layer ℓ is a single-layer MLP—a linear transformation followed by a softmax—producing scores over K experts, with weight matrix $\mathbf{W}_\ell \in \mathbb{R}^{K\times d}$. We define the *expert embedding* for expert k as $\mathbf{v}_\ell^k := \mathbf{W}_\ell[k], k = 1, \ldots, K$, which serves as an anchor vector to compute routing scores. For input x_i, its *routing embedding* is the weighted sum:

$$\mathbf{e}_\ell^{(i)} = \textstyle\sum_{k=1}^{K} g_\ell^{(i,k)} \cdot \mathbf{v}_\ell^k,$$

where $g_\ell^{(i,k)}$ is the routing weight assigned to expert k. Although experts are independently parameterized, the strong correlations between adjacent layers in deep networks [8] and MoEs [14] suggest

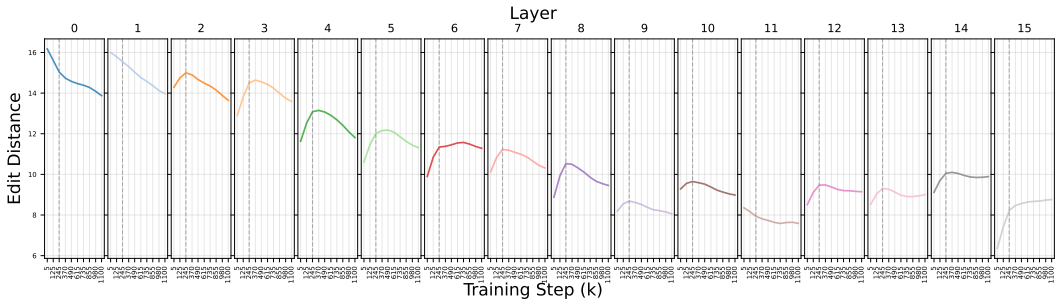

Figure 5: **Layer-wise evolution of pathway edit distance during pretraining** for the data group memorized at $245k$ step. Overall distance decreases from early to later layers, indicating higher complexity in early layers. After pretraining objective convergence, early-layer routing among similar samples simplifies quickly, later layers adapt more gradually, and the final layer diversifies after memorization.

that $\{\mathbf{v}_\ell^k\}$ can be comparable across neighboring layers, supporting structural analysis of routing behavior.

To quantify routing smoothness, we define **pathway consistency** C_i as the normalized average cosine similarity between consecutive layer embeddings for input x_i:

$$C_i = 1 - \frac{1}{L-1} \sum_{\ell=1}^{L-1} \frac{\cos(\mathbf{e}_{i,\ell}, \mathbf{e}_{i,\ell+1})}{\max_\ell \cos(\mathbf{e}_{i,\ell}, \mathbf{e}_{i,\ell+1}) + \epsilon},$$

where $\epsilon = 10^{-8}$ ensures numerical stability. Higher C_i indicates more erratic transitions, while lower values reflect smoother, more consistent routing across layers.

Figure 6 tracks the evolution of pathway consistency and pretraining objective across four domains. A key finding is that even after the objective stabilizes, pathway consistency continues to improve—revealing ongoing refinement in the internal routing of the model. This suggests that smoother, more coherent cross-layer transitions emerge well after memorization, positioning pathway consistency as a valuable lens for understanding generalization beyond what the training objective alone captures.

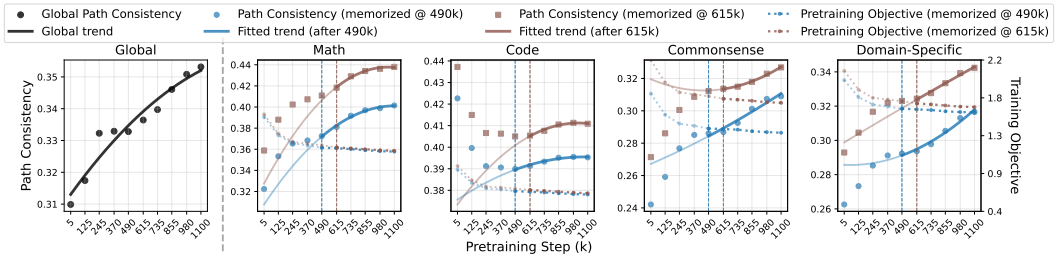

Figure 6: **Pathway consistency vs. training objective** on four domains during pretraining. The global panel (leftmost) shows that the path consistency consistently improves over all data, while each domain panel shows the same trend on two data groups (memorized at $490k$ and $615k$) from each domain after the training objective converges. This reflects a progressively smoother transition of experts between consecutive layers after memorization.

4.3 Pathway Complexity Metrics as Monitors of Generalization

Given the strong link between routing dynamics and generalization, we ask: *To what extent do our pathway metrics predict test performance across domains?* To answer this, we compute Pearson and Spearman correlations between downstream performance and two metrics across pretraining checkpoints. As described in Section 3.1, we apply a lightweight LoRA-based instruction tuning (rank = 32; see Appendix B) solely to equip pretrained checkpoints with basic instruction-following

Table 1: **Correlation between pathway complexity metrics and generalization** (test accuracy) across four domains, with the comparison to training/validation loss. p-value: $p < 0.1$, $p < 0.05$, $p < 0.01$, $p < 0.001$, $p > 0.1$, $p > 0.5$, $p > 0.8$. Preferred correlation: positive, negative.

Metric	Math		Code		Commonsense		Domain-Specific	
	Pearson	Spearman	Pearson	Spearman	Pearson	Spearman	Pearson	Spearman
Pathway Distance (pairwise)	-0.9471	-0.9487	-0.9290	-0.9276	-0.9821	-0.9747	-0.9254	-0.9487
Pathway Consistency (sample-wise)	0.9246	0.9487	0.9786	0.9856	0.9804	0.9747	0.9917	0.9487
Training Loss	-0.0435	-0.2108	-0.0680	-0.2052	0.6427	0.4104	0.7944	0.9487
Training Loss (moving average)	0.4714	0.3162	0.2481	-0.2052	0.7931	0.6669	0.8705	0.9487
Validation Loss	0.4801	-0.1054	0.5466	0.8721	-0.5752	-0.4617	-0.3339	-0.3162

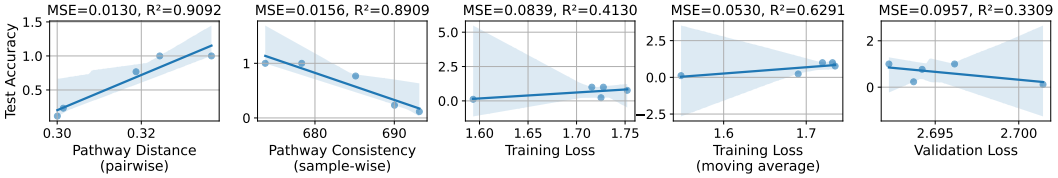

Figure 7: Correlation of the two pathway complexity metrics with benchmark performance. Compared to the training objective and its moving average, both the pathway edit distance and consistency are highly correlated with the test accuracy, though they are also computed on training data.

ability for benchmark evaluation. The robustness of this setup is further supported by the LoRA rank ablation in Appendix E.2. Our correlation analysis focuses on checkpoints after domain-level accuracy begins its sustained rise (shown in Figure 1), thereby isolating the mechanisms that drive generalization beyond the random-accuracy phase.

Table 1 summarizes the correlation results across four tasks. Pathway consistency (sample-wise) shows exceptionally strong positive correlations with test accuracy—often exceeding 0.97 and highly significant—indicating that coherent, stable routing across layers is a key marker of generalization. In contrast, pairwise pathway similarity consistently yields strong negative correlations (around -0.93), suggesting that as generalization improves, inputs tend to follow increasingly similar expert routes. By comparison, as shown in Figure 7, training and validation loss metrics exhibit weaker and less consistent correlations. Notably, while moving-average training loss shows relatively high correlation in certain domains (e.g., Code, Domain-Specific), its direction is inconsistent with expectation: since loss is minimized during training, we would expect a *negative* correlation with test accuracy—not the *positive* one observed here. This mismatch further underscores that conventional pretraining metrics fail to reflect the internal transition from memorization to generalization.

These findings position pathway metrics as robust, domain-general indicators of generalization. Importantly, they rely only on training dynamics—without requiring held-out validation data—making them especially valuable for real-time monitoring in large-scale or resource-constrained settings.

4.4 Theoretical Connections & Explanations

We connect expert routing patterns with model generalization via the *complexity* of routing, formalized through the *effective dimension* of the routing kernel. This effective dimension will serve as the key determinant of MoE generalization.

Concretely, $\boldsymbol{H}^*$ is the Gram matrix of the routing kernel with entries $\boldsymbol{H}^*_{ij} = \Theta_{\text{route}}(x_i, x_j)$, capturing similarity between inputs induced by both routing overlap and expert functions. The effective dimension is defined as $\text{Tr}(\boldsymbol{H}^*(\boldsymbol{H}^* + \lambda I)^{-1})$, measuring the complexity of the function space induced by routing.

Theorem 4.1 (Generalization Bound). *Under NTK regime with K experts, with probability $1 - \delta$ over n samples, where C_1, C_2 are NTK-dependent constants and λ is the regularization parameter:*

$$\mathcal{E} \leq \underbrace{\frac{C_1 \lambda^2}{n} \boldsymbol{y}^\top (\boldsymbol{H}^* + \lambda I)^{-2} \boldsymbol{y}}_{\textit{Bias}} + \underbrace{C_2 \left(\frac{\sigma^2 Tr(\boldsymbol{H}^*(\boldsymbol{H}^* + \lambda I)^{-1})}{n} + \frac{\sigma^2 \log(1/\delta)}{n} \right)}_{\textit{Variance + Noise}}$$

This provides a lens on grokking: as routing evolves from random to structured, the effective dimension collapses, triggering improvements in generalization. Intuitively, this collapse corresponds to a simplification of routing pathways: the kernel spectrum becomes more concentrated, reducing the hypothesis space and mitigating overfitting. The full theoretical setup, assumptions, and proof are detailed in Appendix C, while Appendix C.4 empirically confirms that effective dimension is tightly aligned with routing edit distance, establishing pathway complexity as a key driver of MoE generalization.

5 Conclusion

This paper provides the first empirical evidence that, in large-scale LLM pretraining, generalization can emerge well after memorization has been achieved. Through an analysis of routing dynamics in a 7B-parameter MoE model, we propose two novel pathway-based metrics that accurately track generalization without requiring finetuning or external validation sets. Our findings reveal a transition from memorization to generalization, marked by increased pathway similarity and consistency, and supported by theoretical connections between routing structure and generalization bounds. These insights offer a foundation for more transparent and reliable monitoring of generalization in large-scale models. Future work will extend these pathway-based metrics beyond MoE, by constructing analogous "virtual pathways" in dense models, moving toward a unified framework for tracking generalization in foundation models.

References

[1] Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. Large language models for mathematical reasoning: Progresses and challenges, 2024. URL `https://arxiv.org/abs/2402.00157`.

[2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report, 2023. URL `https://arxiv.org/abs/2309.16609`.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. URL `https://arxiv.org/abs/2005.14165`.

[4] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[5] Branton DeMoss, Silvia Sapora, Jakob Foerster, Nick Hawes, and Ingmar Posner. The complexity dynamics of grokking, 2024. URL `https://arxiv.org/abs/2412.09810`.

[6] Simin Fan, Razvan Pascanu, and Martin Jaggi. Deep grokking: Would deep neural networks generalize better?, 2024. URL `https://arxiv.org/abs/2405.19454`.

[7] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.